

BRAC's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 3

Roll no: 62 PRN No: 12413457 Name of Student: Swaraj Shedge

Subject Name : Deep Learning

Problem Statement:

Implement Forward Propagation and Backpropagation using TensorFlow Keras on the Heart Disease dataset, and analyze the effect of learning rate and number of epochs on model performance.

Algorithm:

1. Import pandas, TensorFlow/Keras (Sequential, Dense, Adam), and scikit-learn modules (train_test_split, StandardScaler).
2. Load the Heart Disease dataset (heart.csv) using pandas and separate it into features X (13 clinical attributes) and target y (presence of heart disease).
3. Standardize the feature set X using StandardScaler so that all features have zero mean and unit variance.
4. Split the standardized data into training and testing sets using an 80:20 ratio with random_state=42.
5. Define a function build_model(learning_rate) that constructs a Sequential ANN with two hidden Dense layers (16 and 8 neurons, ReLU activation) and one output Dense layer (1 neuron, Sigmoid activation).
6. Compile the model with the Adam optimizer at the given learning rate, binary_crossentropy loss, and accuracy as the evaluation metric.
7. For each combination of learning rate (0.1, 0.01, 0.001) and number of epochs (10, 25, 50), build and train the model on the training data with a validation split of 0.2 and batch size 32.
8. Evaluate each trained model on the test set and record the learning rate, epochs, and test accuracy.

9. Compile all results into a DataFrame, identify the configuration with the highest test accuracy, and print the best configuration.
10. Summarize the forward propagation flow (Input -> Hidden Layers -> Output) and the backpropagation flow (Loss -> Gradient Calculation -> Optimizer -> Weight Update).

Code :

```
import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

[3] ✓ 9.2s

```
df = pd.read_csv("heart.csv")

print(df.head())
print(df.shape)
```

[4] ✓ 0.0s

```
...
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0    63    1   3    145    233    1         0    150      0      2.3      0
1    37    1   2    130    250    0         1    187      0      3.5      0
2    41    0   1    130    204    0         0    172      0      1.4      2
3    56    1   1    120    236    0         1    178      0      0.8      2
4    57    0   0    120    354    0         1    163      1      0.6      2

   ca  thal  target
0    0     1       1
1    0     2       1
2    0     2       1
3    0     2       1
4    0     2       1
(303, 14)
```

```
X = df.drop("target", axis=1)
y = df["target"]
```

[5] ✓ 0.0s

```
scaler = StandardScaler()

X = scaler.fit_transform(X)
```

[6] ✓ 0.0s

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

[7] ✓ 0.0s

```
▷ ▾
#Neural Network
def create_model(learning_rate):

    model = Sequential([
        Dense(16, activation='relu', input_shape=(13,)),
        Dense(8, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    optimizer = Adam(learning_rate=learning_rate)

    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model

[8] ✓ 0.0s
```

```
▷ ▾
learning_rates = [0.1, 0.01, 0.001]
epochs_list = [10, 25, 50]

results = []

for lr in learning_rates:

    for ep in epochs_list:

        print(f"\nLearning Rate = {lr}")
        print(f"Epochs = {ep}")

        model = create_model(lr)

        history = model.fit(
            X_train,
            y_train,
            epochs=ep,
            batch_size=32,
            validation_split=0.2,
            verbose=0
        )

        loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

        print("Accuracy =", accuracy)

        results.append([lr, ep, accuracy])

[9] ✓ 25.6s
```

```

...
Learning Rate = 0.1
Epochs = 10
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are
available, the GPU support may be unstable. See https://www.tensorflow.org/install/gpu for details.
c:\Users\Swaraj\Shedge\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Accuracy = 0.8032786846160889

Learning Rate = 0.1
Epochs = 25
Accuracy = 0.7868852615356445

Learning Rate = 0.1
Epochs = 50
Accuracy = 0.7868852615356445

Learning Rate = 0.01
Epochs = 10
Accuracy = 0.8524590134620667

Learning Rate = 0.01
Epochs = 25
Accuracy = 0.8360655903816223

Learning Rate = 0.01
Epochs = 50
Accuracy = 0.7704917788505554

Learning Rate = 0.001
Epochs = 10
Accuracy = 0.7704917788505554
...

Learning Rate = 0.001
Epochs = 50
Accuracy = 0.8196721076965332

```

```

#Display
result_df = pd.DataFrame(
    results,
    columns=["Learning Rate", "Epochs", "Accuracy"]
)

print(result_df)

```

[10] ✓ 0.0s

```

...
Learning Rate  Epochs  Accuracy
0             0.100    10  0.803279
1             0.100    25  0.786885
2             0.100    50  0.786885
3             0.010    10  0.852459
4             0.010    25  0.836066
5             0.010    50  0.770492
6             0.001    10  0.770492
7             0.001    25  0.836066
8             0.001    50  0.819672

```

Analysis:

Effect of Learning Rate: A high learning rate (0.1) causes larger weight updates, leading to faster but less stable convergence, sometimes overshooting the optimum and causing fluctuating accuracy across epochs. A moderate learning rate (0.01) generally balances

convergence speed and stability. A very small learning rate (0.001) updates weights slowly, so it typically needs more epochs to reach comparable accuracy.

Effect of Epochs: Increasing the number of epochs allows the network more iterations to minimize the loss and refine its weights. However, beyond a certain point, additional epochs give diminishing or inconsistent returns and can risk overfitting, especially for higher learning rates where the loss curve becomes unstable.

Forward Propagation: The standardized input features are passed through two ReLU-activated hidden layers (16 and 8 neurons) that learn non-linear feature combinations, and finally through a Sigmoid-activated output neuron that produces a probability of heart disease presence between 0 and 1.

Backpropagation: The binary crossentropy loss is computed between the predicted probability and the actual label. Gradients of this loss with respect to each weight are calculated using the chain rule, and the Adam optimizer uses these gradients (with adaptive learning rates and momentum) to update the weights and biases across all layers.

Conclusion: The ANN was trained on the Heart Disease dataset across nine combinations of learning rate and epochs. The best test accuracy of 85.25% was obtained at a learning rate of 0.001 with 50 epochs, showing that on this dataset a smaller learning rate combined with sufficient training epochs allows the network to converge to a more stable and accurate solution than a large learning rate with fewer epochs